

Question No:9

Question description

A game consists of n rooms and m teleporters. At the beginning of each day, you start in room 1 and you have to reach room n . You can use each teleporter at most once during the game. How many days can you play if you choose your routes optimally?

Constraints

$2 \leq n \leq 500$

$1 \leq m \leq 1000$

$1 \leq a, b \leq n$

Input

The first input line has two integers n and m : the number of rooms and teleporters. The rooms are numbered $1, 2, \dots, n$. After this, there are m lines describing the teleporters. Each line has two integers a and b : there is a teleporter from room a to room b . There are no two teleporters whose starting and ending room are the same.

Output

First print an integer k : the maximum number of days you can play the game. Then, print k route descriptions according to the example. You can print any valid solution.

▼ Logical Test Cases

Test Case 1	Test Case 2
INPUT (STDIN) 6 7 1 2 1 3 2 6 3 4 3 5 4 6 5 6 EXPECTED OUTPUT 2 4 1 3 5 6 3 1 2 6	INPUT (STDIN) 6 7 1 4 3 5 4 6 5 6 2 2 1 3 2 6 EXPECTED OUTPUT 2 4 1 3 5 6 3 1 4 6

Code:

```
1  #include <stdio.h>
2  #include <string.h>
3
4  #define MAXN 505
5  #define MAXE 4005
6
7  typedef struct
8  {
9      int to;
10     int capacity;
11     int rev;
12     int original;
13 } Edge;
14
15 Edge graph[MAXN][MAXE];
16 int degree[MAXN];
17
18 int level[MAXN];
19 int ptr[MAXN];
20
21 int n, m;
22
23 /* Add an edge */
24 void addEdge(int u, int v)
25 {
26     graph[u][degree[u]].to = v;
27     graph[u][degree[u]].capacity = 1;
28     graph[u][degree[u]].rev = degree[v];
29     graph[u][degree[u]].original = 1;
```

```
31     graph[v][degree[v]].to = u;
32     graph[v][degree[v]].capacity = 0;
33     graph[v][degree[v]].rev = degree[u];
34     graph[v][degree[v]].original = 0;
35
36     degree[u]++;
37     degree[v]++;
38 }
39
40 /* BFS for Dinic */
41 int bfs(int source, int sink)
42 {
43     int queue[MAXN];
44     int front = 0, rear = 0;
45
46     for (int i = 1; i <= n; i++)
47         level[i] = -1;
48
49     level[source] = 0;
50     queue[rear++] = source;
51
52     while (front < rear)
53     {
54         int u = queue[front++];
```

```

56         for (int i = 0; i < degree[u]; i++)
57         {
58             Edge *e = &graph[u][i];
59
60             if (e->capacity > 0 && level[e->to] == -1)
61             {
62                 level[e->to] = level[u] + 1;
63                 queue[rear++] = e->to;
64             }
65         }
66     }
67
68     return level[sink] != -1;
69 }
70
71 /* DFS for Dinic */
72 int dfs(int u, int sink, int flow)
73 {
74     if (u == sink)
75         return flow;
76
77     for (int *i = &ptr[u]; *i < degree[u]; (*i)++)
78     {
79         Edge *e = &graph[u][*i];
80
81         if (e->capacity > 0 &&

```

```

82         level[e->to] == level[u] + 1)
83     {
84         int newFlow = flow < e->capacity
85             ? flow
86             : e->capacity;
87
88         int pushed = dfs(e->to, sink, newFlow);
89
90         if (pushed > 0)
91         {
92             e->capacity -= pushed;
93             graph[e->to][e->rev].capacity += pushed;
94
95             return pushed;
96         }
97     }
98 }
99
100     return 0;
101 }
102
103 /* Maximum flow */
104 int maxFlow(int source, int sink)

```

```
105 {
106     int flow = 0;
107
108     while (bfs(source, sink))
109     {
110         for (int i = 1; i <= n; i++)
111             ptr[i] = 0;
112
113         while (1)
114         {
115             int pushed = dfs(source, sink, 1000000000);
116
117             if (pushed == 0)
118                 break;
119
120             flow += pushed;
121         }
122     }
123
124     return flow;
125 }
126
127 /*
128     Find one path using edges that currently
129     carry flow.
```

```

130  */
131  int findPath(int u, int sink, int visited[],
132             int path[], int *len)
133  {
134      if (u == sink)
135      {
136          path[*len++] = u;
137          return 1;
138      }
139
140      visited[u] = 1;
141
142      for (int i = 0; i < degree[u]; i++)
143      {
144          Edge *e = &graph[u][i];
145
146          /*
147           Original edge with capacity 0 means
148           one unit of flow is using this edge.
149          */
150          if (e->original == 1 &&
151              e->capacity == 0 &&
152              !visited[e->to])
153          {
154              if (findPath(e->to, sink, visited,
155                          path, len))

```

```
156         {
157             path[(*len)++] = u;
158             return 1;
159         }
160     }
161 }
162
163 return 0;
164 }
165
166 int main()
167 {
168     scanf("%d %d", &n, &m);
169
170     for (int i = 1; i <= n; i++)
171         degree[i] = 0;
172
173     /* Read teleporters */
174     for (int i = 0; i < m; i++)
175     {
176         int a, b;
177         scanf("%d %d", &a, &b);
178
179         addEdge(a, b);
```

```
180     }
181
182     /*
183     |     Find maximum number of edge-disjoint
184     |     paths from 1 to n.
185     */
186     int k = maxFlow(1, n);
187
188     printf("%d\n", k);
189
190     /*
191     |     Extract the k routes.
192     */
193     for (int route = 0; route < k; route++)
194     {
195         int visited[MAXN] = {0};
196         int path[MAXN];
197         int len = 0;
198
199         findPath(1, n, visited, path, &len);
200
201         /*
202         |     Path is stored backwards:
203         |     n ... 3 2 1
204
205         |     Print it in forward order.
206         */
207         printf("%d\n", len);
208     }
```

```

208
209     for (int i = len - 1; i >= 0; i--)
210     {
211         printf("%d", path[i]);
212
213         if (i > 0)
214             printf(" ");
215     }
216
217     printf("\n");
218
219     /*
220      Remove the flow from this path so
221      the same teleporter isn't used again.
222     */
223     for (int i = len - 1; i > 0; i--)
224     {
225         int u = path[i];
226         int v = path[i - 1];
227
228         for (int j = 0; j < degree[u]; j++)
229         {
230             Edge *e = &graph[u][j];
231
232             if (e->original == 1 &&
233                 e->to == v &&
234                 e->capacity == 0)
235             {
236                 e->capacity = 1;
237                 break;
238             }
239         }
240     }
241
242     return 0;
243
244 }

```

Output:

Test Case-1:

```
6 7
1 2
1 3
2 6
3 4
3 5
4 6
5 6
2
3
1 2 6
4
1 3 4 6
```

Test Case-2:

```
6 7
1 4
3 5
4 6
5 6
2 2
1 3
2 6
2
3
1 4 6
4
1 3 5 6
PS C:\Users\kesha\OneDrive\Documents\3sem\DSA\D
```

Keshav Kumar Rauniyar

CH.SC.U4CSE25236